PASO 01 · INTEGRANTE 2

Conectar el motor con la API

Guía de trabajo para el integrante 2. Proyecto SSO - Multiprogramación por lotes. Windows, PowerShell y GitHub. Revisión: 12 de octubre de 2026.

Tu tarea

Vas a crear el puente entre el motor de Python y la pantalla. La API recibirá solicitudes para crear una simulación, consultar su estado y aplicar I/E/P/C. También harás que el servidor avance el reloj una vez por segundo.

Parte	Estado actual y responsable
simulation/	Integrante 1. Motor terminado: generación, lotes de hasta 3, operaciones, tiempos e I/E/P/C.
backend/main.py	Integrante 2. FastAPI ya arranca; existe GET /api/health y se sirve frontend/. Faltan las rutas de simulación.
frontend/	Integrante 3. Hay una página inicial que consulta health. Faltan los paneles y los controles finales.
docs/ y tests/	Contrato JSON, entrega del motor, fixture real y pruebas. Úsalos para mantener compatibles las tres partes.

Cómo se conecta todo

```
Pantalla -> rutas API -> servicio -> SimulationEngine
Pantalla <- JSON      <- snapshot <- estado del motor
                            ^
                  reloj del servidor
```

El motor calcula y conserva el estado. El servicio guarda los motores por ID y organiza su acceso. Las rutas reciben y responden JSON. La pantalla dibuja la respuesta que reciba.

Punto de partida comprobado

El motor está incluido en main de GitHub al preparar esta guía (commit a67b9ba). Sus 41 pruebas pasan. El README todavía contiene frases anteriores a la entrega del motor: actualízalas al terminar tu API. Las instrucciones de las siguientes páginas describen el trabajo pendiente; esos archivos aún no están creados.

Ruta de trabajo: iniciar sesión → clonar → crear rama → preparar Python → implementar → probar → commit → push → PR.

PASO 02 · INTEGRANTE 2

Preparar Git e iniciar sesión

Abre PowerShell. Estos pasos se hacen una vez por computadora. Debes tener una cuenta de GitHub y aceptar la invitación al repositorio como colaborador.

1. Revisar las herramientas

```
git --version
gh --version
py -3.12 --version
```

Git guarda las versiones de tus archivos. GitHub las comparte con el equipo. gh es GitHub CLI: permite iniciar sesión y crear el PR desde la terminal. El proyecto usa Python 3.12.x.

Si falta alguna herramienta, instala Git para Windows, GitHub CLI o Python 3.12. Cierra y vuelve a abrir PowerShell después de instalarlas. Si ya están disponibles, sigue al paso 2.

2. Iniciar sesión en GitHub

```
gh auth login --hostname github.com --git-protocol https --web
gh auth setup-git
gh auth status
```

La terminal mostrará un código y abrirá el navegador. Copia el código, entra con tu propia cuenta y autoriza GitHub CLI. Si pregunta si deseas autenticar Git con tu cuenta, elige Yes. Al final, auth status debe mostrar tu usuario conectado. Ayuda oficial.

3. Poner tu nombre en los commits

```
git config --global user.name "TU NOMBRE"
git config --global user.email "TU CORREO DE GITHUB"
```

Reemplaza los textos entre comillas por tus datos. Para el correo también puedes usar la dirección privada noreply que aparece en GitHub → Settings → Emails. Esta configuración identifica al autor del commit; el inicio de sesión lo hiciste con gh.

Antes de continuar

Abre el repositorio del equipo con tu cuenta. Si no tienes acceso de escritura, pide al dueño que te agregue y acepta la invitación. La guía usa una rama dentro del repositorio compartido.

PASO 03 · INTEGRANTE 2

Clonar, crear rama y arrancar

Clonar descarga una copia del proyecto. Una rama permite trabajar en tu API sin mezclar todavía tus cambios con main. Ejecuta los comandos en el orden indicado.

1. Descargar el proyecto una sola vez

```
New-Item -ItemType Directory -Force "$HOME/Documents/Proyectos"
Set-Location "$HOME/Documents/Proyectos"
git clone https://github.com/iLaurx/sso-multiprogramacion.git
Set-Location sso-multiprogramacion
```

Si ya lo clonaste, entra a tu carpeta existente y omite git clone. Desde aquí, todos los comandos se ejecutan en la raíz del proyecto, donde está requirements.txt.

2. Actualizar main y crear tu rama

```
git status
git switch main
git pull --ff-only origin main
git switch -c feature/i2-api-simulacion
git branch --show-current
```

Antes de cambiar de rama, status debe indicar que no hay cambios pendientes. La última línea debe mostrar feature/i2-api-simulacion. Si la rama ya existe, usa git switch feature/i2-api-simulacion en lugar de crearla otra vez.

3. Preparar tu propio entorno de Python

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

Crea .venv solo si no tienes un entorno de Python 3.12 en esta copia. Cada integrante tiene el suyo. Con la base revisada, pytest debe aprobar 44 casos: 41 del motor y 3 de arranque. Si falla, resuelve el fallo antes de agregar tu código.

Abre http://127.0.0.1:8000/: debe mostrar API: ok. Abre http://127.0.0.1:8000/docs: por ahora aparece health. Detén el servidor con Ctrl+C. Abre la carpeta en VS Code con code . o Archivo → Abrir carpeta.

Comprueba que existe simulation/engine.py. Si no aparece tras actualizar main, consulta al integrante 1 antes de empezar: necesitas su motor integrado.

PASO 04 · INTEGRANTE 2

Qué leer y qué archivos crear

Primero identifica las piezas. Después crea cada archivo cuando llegues a su implementación; no necesitas llenar el proyecto de archivos vacíos.

Lee estos archivos en este orden

1. docs/entrega-motor.md: cómo importar y usar el motor.
2. docs/contrato-api.md: rutas, entradas, respuestas y errores.
3. backend/main.py: arranque y publicación de la página.
4. tests/test_health.py: ejemplo de pruebas con TestClient.

Archivo	Qué vas a hacer
Crear backend/schemas.py	Validar la cantidad, la semilla y la acción con modelos de Pydantic.
Crear backend/service.py	Guardar motores por ID, un bloqueo por motor y las tareas que avanzan el reloj. Crear, consultar y aplicar acciones.
Crear backend/routes.py	Definir POST create, GET estado y POST actions mediante APIRouter. Traducir errores del servicio a HTTP.
Modificar backend/main.py	Conectar el servicio, registrar el router y manejar el inicio/apagado con lifespan. Conservar health y la página.
Crear tests/test_api.py	Probar rutas, validaciones, errores y que consultar no avance el reloj.
Crear tests/test_service.py	Probar ticks controlados, pausa, final, bloqueo y cierre de tareas.
Actualizar README.md	Describir motor/API ya implementados y cómo ejecutar/probar. Ajustar el contrato solo si el equipo acuerda cambios.

Primer contacto con el motor

```
from simulation import SimulationEngine, SimulationStateError

motor = SimulationEngine.create(numero_trabajos=4, seed=123)
print(motor.snapshot())
motor.tick()
print(motor.snapshot()["reloj"])  # 1
```

Este ejemplo es para entenderlo: el motor no espera un segundo real. tick() avanza una unidad cuando tú lo llamas. En tu entrega, el servicio decidirá cuándo llamarlo.

Tu alcance principal es backend/ y sus pruebas. El integrante 1 mantiene simulation/; el integrante 3 construye la pantalla. Si detectas un error en otra parte, coordínalo con su responsable.

PASO 05 · INTEGRANTE 2

Entradas, rutas y respuestas

Empieza por schemas.py y las tres rutas de simulación. Conserva los nombres del contrato para que el integrante 3 pueda conectarse sin cambiar su pantalla.

1. Validar la entrada en schemas.py

```
from typing import Annotated, Literal
from pydantic import BaseModel, Field, StrictInt

class CrearSimulacion(BaseModel):
    numero_trabajos: Annotated[StrictInt, Field(gt=0)]
    seed: StrictInt | None = None

class AccionSimulacion(BaseModel):
    accion: Literal["interrupt", "error", "pause", "continue"]
```

numero_trabajos admite un entero positivo. Rechaza 0, negativos, true, 2.5, 2.0 y "4". seed es opcional: admite entero o null. FastAPI devuelve 422 cuando estos modelos rechazan la entrada. Tipos estrictos de Pydantic.

Ruta	Qué debe hacer
POST /api/simulations	Entrada {"numero_trabajos":4,"seed":123}. Crear motor e ID. Responder 201 y snapshot completo con reloj 0.
GET /api/simulations/{id}	Consultar ese motor. Responder 200 y snapshot completo. Solo lee; no llama tick().
POST /api/simulations/{id}/actions	Entrada {"accion":"pause"}. Aplicar la acción. Responder 200 y snapshot completo actualizado.

2. Mantener el JSON del motor

```
respuesta = {"simulacion_id": simulacion_id, **motor.snapshot()}
```

Haz esta copia dentro del bloqueo del servicio. Conserva estado, reloj, lote_actual, lotes_pendientes, total_trabajos, en_ejecucion, en_espera y terminados. Conserva también todos los campos de cada proceso. El ID externo lo agrega la API.

3. Responder los errores correctos

422: entrada o acción inválida. 404: ID inexistente. 409: acción incompatible, capturando SimulationStateError y usando su mensaje en detail. Por ejemplo: HTTPException(status_code=409, detail=str(error)).

I/E en pausa o al terminar son 409. P y C al terminar también son 409. P en paused y C en running son válidos y dejan el estado igual. Devuelve {"detail":"mensaje"} para errores controlados; la validación puede devolver detail como lista.

PASO 06 · INTEGRANTE 2

El servicio une las piezas

En service.py concentra el acceso al motor. Así las rutas se limitan a recibir datos, llamar al servicio y responder. Esta es una organización propuesta para tu implementación.

1. Guardar cada simulación

Usa un diccionario en memoria. La clave es un ID nuevo, por ejemplo str(uuid.uuid4()). Cada registro guarda su SimulationEngine, un asyncio.Lock y la tarea del reloj. Usa funciones async en el servicio y en las rutas para compartir estos bloqueos.

2. Separar las operaciones

Método sugerido	Responsabilidad
create(numero_trabajos, seed)	Construir el motor, generar el ID, guardar el registro y devolver la respuesta inicial. Si el reloj está habilitado, crear su tarea.
get_snapshot(id)	Buscar el registro o indicar que no existe. Tomar su bloqueo y devolver ID + snapshot.
apply_action(id, accion)	Buscar el registro, tomar el bloqueo, aplicar el método del motor y devolver ID + snapshot sin soltar antes el bloqueo.
tick_once(id)	Buscar, tomar el mismo bloqueo y llamar motor.tick(). Es uso interno y de pruebas.
close()	Cancelar y esperar las tareas del reloj al cerrar el servidor. Liberar los registros de ese servicio.

3. Traducir la acción a una llamada

```
"interrupt" -> motor.interrupt()  # tecla I
"error"     -> motor.fail()       # tecla E
"pause"     -> motor.pause()      # tecla P
"continue"  -> motor.resume()     # tecla C
```

Los métodos devuelven None. Después llama a snapshot() para responder. I y E no agregan segundos; el motor ya conserva los tiempos y decide qué proceso sigue.

4. Usar siempre el mismo bloqueo

```
# Ejemplo dentro de un método async del servicio:
async with registro.lock:
    registro.motor.pause()
    return {"simulacion_id": id, **registro.motor.snapshot()}
```

Bloqueo significa que una sola operación entra a ese motor a la vez. tick, acciones y lecturas deben compartirlo. Así la pantalla no recibe un estado tomado a mitad de un cambio. Evita leer o modificar atributos privados del motor.

PASO 07 · INTEGRANTE 2

Reloj y arranque del servidor

Primero conecta crear, consultar y actuar. Cuando esas operaciones funcionen, agrega el reloj automático y su cierre. No publiques una ruta de tick.

1. Un reloj por simulación, manejado por el servicio

Una opción sencilla es crear una tarea asyncio para cada motor. Antes del primer tick, espera un segundo con await asyncio.sleep(1). Luego toma el bloqueo, llama tick() y repite. Cada tarea avanza solo su motor.

```
# Esquema de la tarea, no un archivo completo:
while True:
    await asyncio.sleep(1)
    async with registro.lock:
        if registro.motor.status == "completed":
            break
        registro.motor.tick()
```

En pausa, tick() no cambia nada. Cuando el motor llega a completed, la tarea termina y el snapshot final queda disponible. GET y las acciones no deben crear tareas adicionales. Evita time.sleep() dentro de funciones async porque detiene otras solicitudes.

2. Permitir pruebas sin reloj real

El servicio debe aceptar una opción auto_clock=False para pruebas. En ese modo no crea tareas automáticas: las pruebas avanzan con tick_once(id). La aplicación normal usa auto_clock=True. Esta opción es interna; no se envía desde la pantalla.

3. Conectar todo en main.py

Usa lifespan de FastAPI para crear un servicio por instancia de aplicación y guardarlo, por ejemplo, en app.state.service. Las rutas lo obtienen desde request.app.state.service. Al apagar, llama await service.close(): cancela y espera todas las tareas pendientes. Lifespan de FastAPI.

Puedes organizarlo con una fábrica create_app(auto_clock=True) y dejar app = create_app() para Uvicorn. Las pruebas podrán usar create_app(auto_clock=False) y tendrán un servicio nuevo, sin compartir simulaciones entre casos.

```
app.include_router(router)  # rutas /api primero
app.mount("/", StaticFiles(directory=frontend, html=True),
          name="frontend")  # montaje de la página al final
```

Conserva health. El montaje de / debe quedar después del router: si lo colocas primero, puede interceptar las solicitudes de la API. Ejecuta un solo worker. En la demo final omite --reload: al reiniciar el servidor se pierden las simulaciones en memoria.

PASO 08 · INTEGRANTE 2

Comprobar que ya conecta

Prueba con el motor real antes de trabajar con la pantalla final. /docs permite enviar solicitudes sin escribir todavía el JavaScript del integrante 3.

1. Recorrido manual en /docs

```
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

Abre http://127.0.0.1:8000/docs. En cada ruta usa Try it out → rellena la entrada → Execute. Copia el simulacion_id que devuelva la creación y úsalo en las demás rutas.

Prueba	Resultado esperado
Crear con 4 trabajos y seed=123	201, estado running, reloj 0, lote 1 y un lote pendiente. El primer proceso es ID 1, TME 7, operación -77 * 97.
Consultar después de unos segundos	200. El reloj avanza por el servidor. Consultar muchas veces no debe acelerarlo.
Enviar {"accion":"pause"}	200, paused. Esperar y consultar: el reloj y los tiempos permanecen iguales.
Enviar interrupt durante pausa	409. Luego enviar continue: 200, running, y vuelve a avanzar.
Probar interrupt y error en running	I reencola conservando tiempo; E termina el actual con estado error, resultado null y restante 0.
Cantidad 0, decimal, true o acción desconocida	422. Consultar un ID inventado devuelve 404.
Dejar terminar una simulación de 1 trabajo	completed, actual y lote null, espera vacía y terminados conservados. P/C/I/E devuelven 409.

2. Pruebas que debes agregar

En test_api.py verifica 201/200/422/404/409 y todos los campos del JSON. Con reloj automático desactivado, varios GET deben conservar el mismo reloj. Avanza desde el servicio y confirma que el siguiente GET refleja el tick.

En test_service.py verifica pausa, continuación, final, dos operaciones concurrentes sin corromper estado y cierre de tareas. Usa un reloj/espera controlable para probar el ciclo automático sin depender de largos sleep reales. Usa TestClient como contexto: with TestClient(app) as client.

```
.\.venv\Scripts\python.exe -m pytest -q
```

Debe pasar toda la suite, incluidas las pruebas anteriores. Entrega al integrante 3 los tres endpoints, un ID creado y el JSON real. El fixture motor-inicial.json es una salida real; contrato-api.json es ilustrativo.

PASO 09 · INTEGRANTE 2

Guardar, subir y abrir tu PR

Un commit guarda una versión local. Push envía tu rama a GitHub. El pull request (PR) propone integrar esos cambios a main para que otro integrante los revise.

1. Revisar y guardar la entrega

```
git branch --show-current
git status
git diff
.\.venv\Scripts\python.exe -m pytest -q
git add backend tests README.md
git diff --cached
git commit -m "feat: conectar motor de lotes con API"
```

Confirma que estás en feature/i2-api-simulacion. Revisa que solo incluyes tus cambios. Si el equipo acordó un cambio en docs/contrato-api.md, agrégalo expresamente. .venv, cachés y credenciales quedan fuera del commit.

2. Publicar tu rama

```
git push -u origin feature/i2-api-simulacion
```

La primera vez usa -u. Los siguientes cambios en esta misma rama se suben con git push.

3. Escribir la descripción y crear el PR

Ejecuta el siguiente bloque en PowerShell. La descripción se guarda temporalmente fuera del proyecto. Ajusta la sección de pruebas para indicar lo que realmente verificaste.

```
$prBodyPath = Join-Path $env:TEMP "pr-api-i2.md"
@'
Conecta el motor existente con las rutas de simulación.
Incluye validación, estado por ID, reloj y bloqueos.

Pruebas realizadas:
- Suite completa de pytest aprobada.
- Crear, consultar e I/E/P/C comprobados en /docs.
- GET no avanza el reloj con auto_clock desactivado.

Solicito revisión del integrante 1 antes de integrar.
'@ | Set-Content -Encoding utf8 $prBodyPath
gh pr create --base main --head feature/i2-api-simulacion `
  --title "Conectar motor de lotes con API" `
  --body-file $prBodyPath
gh pr view --web
```

La terminal devuelve el enlace del PR. Compártelo con el integrante 1 y agrega su cuenta en Reviewers desde GitHub. El PR propone los cambios; todavía falta su revisión e integración. Ayuda oficial de PR.

Si te piden ajustes: modifica, prueba, git add, git commit y git push. El mismo PR se actualiza automáticamente; no abras otro para la misma entrega.

PASO 10 · INTEGRANTE 2

Seguir trabajando sin enredarte

Conserva esta página como ayuda rápida. Si aparece un error, lee el mensaje y ubica el caso antes de repetir comandos.

Al volver al proyecto

```
Set-Location "$HOME/Documents/Proyectos/sso-multiprogramacion"
git switch feature/i2-api-simulacion
git status
git pull --ff-only origin feature/i2-api-simulacion
```

El último comando se usa después de publicar tu rama. Si main recibió cambios: primero guarda tu trabajo en un commit; luego ejecuta git fetch origin y git merge origin/main estando en tu rama. Prueba otra vez antes de subir.

Lo que aparece	Qué hacer
gh o git no se reconoce	Instalar la herramienta y volver a abrir PowerShell.
403 / Permission denied al subir	Revisar gh auth status y aceptar la invitación como colaborador.
La carpeta ya existe al clonar	Entrar en la copia que ya tienes. No volver a clonarla dentro de ella.
La rama ya existe	Usar git switch feature/i2-api-simulacion.
Not a git repository	Entrar en la raíz de sso-multiprogramacion.
No encuentra Python 3.12	Instalar 3.12; no cambiar de versión por tu cuenta.
PowerShell bloquea Activate.ps1	Usar .venv/Scripts/python.exe como en la guía; no necesitas activar.
404 en las rutas nuevas	Revisar que el router se registró antes de app.mount("/").
Conflictos al unir main	Revisar los archivos indicados con el integrante responsable. Para cancelar ese merge: git merge --abort.
Push rechazado porque tu rama avanzó	git fetch origin y git merge origin/feature/i2-api-simulacion. Resolver, probar y subir; evitar force push.

Tu entrega está lista cuando…

Puedes crear, consultar y controlar el motor por API; el reloj lo lleva el servidor; las entradas y errores respetan el contrato; todas las pruebas pasan; health y la página siguen funcionando; el README está actualizado y tu PR permite revisar los archivos cambiados.

Referencias utilizadas

Código y contrato del repositorio revisados el 12/10/2026. Documentación oficial: inicio de sesión, crear PR, lifespan y tipos de Pydantic. Los enlaces son clicables.